

Python & Async Foundations

Stage 0.5 — plain-English concepts, analogies & diagrams. This page grows as we learn.

L1 Async & the Event Loop

how one worker serves thousands

1 The core idea

Your app spends most of its life **waiting** — for the database, the internet, the AI. **Async** is the trick of doing other useful work *during those waits*, using just **one worker**.

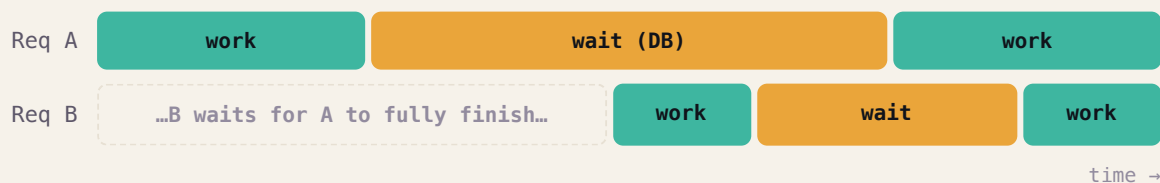
📺 ANALOGY — ONE WAITER, MANY TABLES

A restaurant with **one waiter**. The slow way: take Table 1's order, then *stand in the kitchen* until it's cooked before touching Table 2. The smart way: hand Table 1's order to the kitchen and **immediately** serve Table 2 while it cooks; deliver when the bell rings. One waiter, many tables — because the *cooking* (waiting) is the slow part, not the waiter.

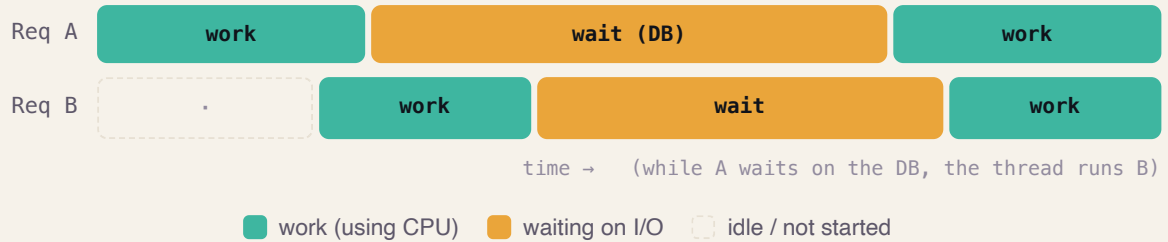
Waiter = one thread · **Tables** = requests · **Kitchen cooking** = I/O wait · “**serve another table while it cooks**” = `await`.

2 Sync vs Async — see the difference

SYNCHRONOUS — THE WORKER SITS IDLE DURING EVERY WAIT (SLOW)



ASYNCHRONOUS — ONE THREAD OVERLAPS THE WAITS (FAST)



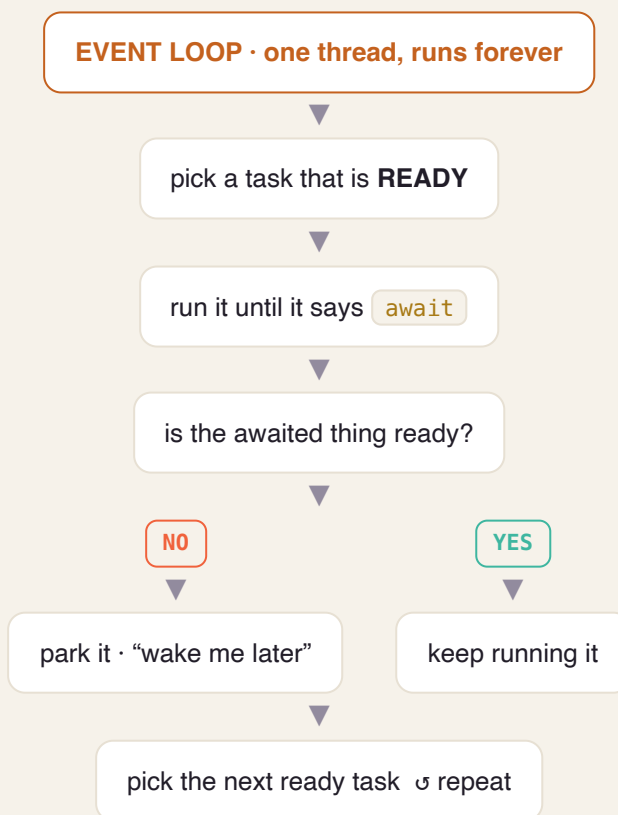
💡 KEY INSIGHT

Async doesn't make one task faster — it stops the worker from **sitting idle**, so many tasks finish in the same wall-clock time.

3 What is the “event loop”?

The **event loop** is the waiter's brain: a `while True:` that runs forever asking one question — “*who's ready for their next step?*” — and running whoever is ready.

THE EVENT LOOP CYCLE



4 What happens at `await` ?

`await` means: “this might take a while — pause me, go help others, wake me when my result is ready.”

AWAIT = PAUSE HERE, RESUME LATER

YOUR TASK

runs → hits `await db.fetch()` → “not ready, park me” →  → woken with result → continues

EVENT LOOP (MEANWHILE)

runs **other tasks** while the DB works → DB result arrives → resumes your task where it paused

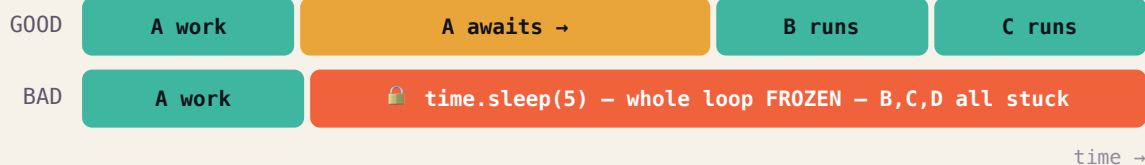
💡 THE SUBTLE PART (BITES PEOPLE LATER)

Between two `await` s your code runs **uninterrupted** — nobody touches your variables. But **across an `await`**, the world can change (other tasks ran while you were paused). This is the seed of the “async race conditions” we’ll handle in later stages.

5 The #1 sin: blocking the loop

Because there's **one** worker, if a task refuses to `await` and does something slow *synchronously*, the whole app **freezes**.

POLITE WAITING VS BLOCKING THE ONE THREAD



🚫 THE RULE

In async code, never call a **blocking** function. Use the async version:

`time.sleep()` ❌ → `await asyncio.sleep()` ✅ · `requests.get()` ❌ → `await httpx.get()` ✅ · sync DB driver ❌ → **async** driver ✅ · heavy CPU work → push to a separate worker pool.

ANALOGY

Blocking = the waiter walks into the kitchen and *personally cooks* one dish while every other table sits ignored.

6 Async is NOT parallelism

Everything above happens on **one worker, one CPU core**. Async overlaps *waiting*, not *computing*.

TWO DIFFERENT TOOLS FOR TWO DIFFERENT PROBLEMS

ASYNC = CONCURRENCY

1 waiter, many tables. Overlaps **WAITS**. → Use for **I/O-bound** work (DB, network, AI). *This is your whole platform.*

PARALLEL = MANY COOKS

Many workers/cores. Overlaps **COMPUTING**. → Use for **CPU-bound** work (heavy math). Needs multiple **processes**.

YOUR DEVOPS BRIDGE

This is exactly how **nginx / Envoy** serve tens of thousands of connections on a few threads with an epoll/kqueue event loop. And “blocking the loop” is the same “one bad call wedged the whole pod” incident you've debugged — now you know the mechanism.

CHEAT-SHEET — THE WHOLE LESSON IN 8 LINES

- App spends most time **waiting** → async fills those waits with other work.
- 1 thread + event loop = one waiter serving many tables.
- `await` = “pause me, help others, wake me when ready.”
- Between awaits: uninterrupted. Across an await: the world may have changed.
- A blocking call on the loop = the whole app freezes (cardinal sin).
- Fix: always use async versions (`asyncio.sleep`, `httpx`, async DB driver).
- async = concurrency (overlap **waits**), NOT parallelism (overlap **CPU**).
- I/O-bound → async. CPU-bound → processes.

✓ SELF-CHECK

1. In the waiter analogy, what is the “kitchen cooking,” and what is `await` ?
2. Why does async help an app that talks to a DB and an API, but not one doing heavy math?
3. A teammate writes `time.sleep(3)` in an async function. What happens to the other 500 users, and what should they have written?

🕒 MY ANSWERS & FEEDBACK

Understanding 3/3 · Precision ~9/10

✓ Q1 · What is “kitchen cooking,” and what is `await` ?

My answer: Kitchen cooking = I/O wait (DB / network / AI calls). `await` = tell the waiter (thread) not to idle while food cooks — go take other orders.

Correct. Precision: `await` is the point where the task hands control *back to the event loop* — that's what frees it to serve others.

⚠️ Q2 · Why does async help I/O apps but not heavy math?

My answer: One thread never idles, handling several tasks at once; heavy math needs multiple processors (multiple cooks).

Right conclusion — one word to fix: async is **concurrent** (interleaved on ONE thread — one thing at a time, overlapping the *waits*), **not parallel**. Heavy math is CPU-bound: the CPU is busy the whole time so there are *no waits to overlap*, and Python's GIL runs one thread at a time anyway → you need multiple **processes**.

✓ Q3 · A teammate writes `time.sleep(3)` in an async function.

My answer: The one thread freezes for 3s; B, C, D... all wait. Use `asyncio.sleep(3)` so the request pauses without holding the thread.

Correct. Precision: `asyncio.sleep` *yields to the event loop*, which is why the other ~500 users keep being served during those 3s.

1 How a Python variable really works

A variable name is just a **sticky tag you put on a box (an object)**. The *type lives on the box, not the tag* — and you can peel the tag off and stick it on a different box anytime.

A VARIABLE IS A TAG YOU STICK ON AN OBJECT

x —tag—▶ [int · 5]

▼ reassign — just move the tag

x —tag—▶ [str · "hi"]

This is **dynamic typing**. Breezy for scripts, but risky at scale: a wrong-type value rides silently through five functions and explodes far away. Application Python tames this with **types**.

2 Type hints are *labels* — ignored at runtime

You label functions: `def add(a: int, b: int) -> int`. But the Python interpreter **does not enforce** them — `add("x", "y")` still runs.

📦 ANALOGY

A type hint is like writing “**FRAGILE — GLASS**” on a box. The label doesn't physically stop someone packing a brick — it's guidance for **humans and tools**. Two “guards” read those labels for you 🙌

3 Guard #1 — `mypy` (the inspector, before the flight)

`mypy` reads your labels and checks your code **before it runs**: `add("x", 1)` → **X**
expected int, got str.

✈️ ANALOGY

mypy = the airport inspector who X-rays every labeled box at check-in — catching “you said glass but packed a brick” *before* the plane departs, not at 30,000 ft (3am in prod). Put it in CI and a whole class of bugs dies at dev time.

THE TYPING WORDS YOU'LL USE

`list[int]`, `dict[str, float]` · `str | None` (optional) ·

`Literal["pending", "running", "done"]` (only these exact values) · `Protocol`

(“anything with these methods fits” — for ports/adapters).

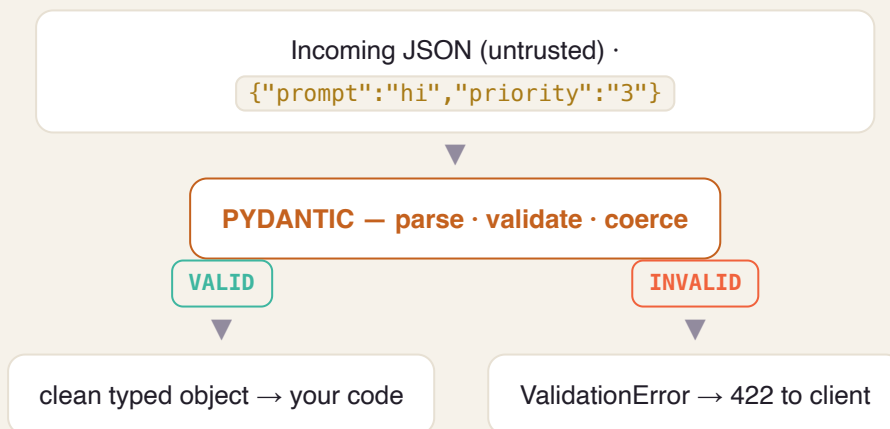
4 Guard #2 — Pydantic (customs, at the border)

Hints don't check **incoming data**. When a client sends `{"priority": "abc"}`, plain Python won't stop it landing in an int field. At a **trust boundary** (API request, config, third-party data) you need **runtime** checking — that's **Pydantic v2**: it parses, validates, and coerces, and rejects bad input loudly.

ANALOGY

Pydantic = the customs officer at the border — opens each incoming bag, checks it matches the declared contents, converts where sensible, and loudly rejects what's wrong. (v2's core is Rust → fast; **FastAPI uses it automatically** → clean `422` for free.)

HOW A REQUEST ENTERS YOUR APP



5 The pattern: validate at the edges, trust the inside

TWO ZONES, TWO GUARDS

EDGE — UNTRUSTED

API requests, config files, third-party responses → **Pydantic validates here, once.**

CORE — TRUSTED

Already-validated typed objects flow freely → **mypy checks your code**; no repeated runtime validation.

ANALOGY

Airport security screens you **once** at the entrance; inside the secure zone you're trusted — nobody re-screens at every gate. Pydantic at the doors, plain typed objects inside.

YOUR DEVOPS BRIDGE

This is **JSON Schema / OpenAPI request validation / Terraform variable type constraints** — the same idea, now living inside your Python at its boundary.

CHEAT-SHEET

- A variable is a tag; the type lives on the **object**, not the name (dynamic typing).
- Type hints are labels — the interpreter **ignores** them at runtime.
- **mypy** = static inspector: catches type bugs **before** you run (put it in CI).
- **Pydantic** = runtime customs at the boundary: parse, coerce, reject bad input.
- Use **both**: mypy for your code, Pydantic for untrusted input.
- Pattern: **validate at the edges, trust the typed core.**
- Handy types: `list[int]`, `str | None`, `Literal[...]`, `Protocol`.
- FastAPI uses Pydantic automatically → clean 422 on bad requests.

✓ SELF-CHECK

1. Is a Python type attached to the *name* or the *object*? Why does that make hints “not enforced” at runtime?
2. You receive a webhook from Stripe. Which guard applies — mypy, Pydantic, or both — and *where exactly* does each one act?
3. Why is `status: Literal["pending", "running", "done"]` safer than `status: str`?

1 The idea: a function that can pause

Normal functions run start-to-finish and forget everything. A **generator** can **pause in the middle, hand back a value, and later resume exactly where it left off** — remembering all its variables.

ANALOGY – A PEZ DISPENSER

A normal function dumps the whole bag of candy at once. A generator is a **Pez dispenser**: press once → one candy (`yield`), it pauses; press again → the next one. It produces values **one at a time, on demand (lazily)**, so it never has to hold the whole bag in memory.

YIELD = PAUSE & HAND BACK A VALUE

call generator → runs to first `yield 1` → **pauses**, returns 1

▼ ask for next

resumes after the yield → runs to `yield 2` → pauses, returns 2

▼ ...until it finishes

2 Coroutines = the same trick, for `async`

An `async def` function is a **coroutine** — it uses the *exact same “pause & resume” machinery* as a generator. The only difference: a generator pauses at `yield` to hand back **data**; a coroutine pauses at `await` to hand back **control** to the event loop (Lesson 1!).

KEY INSIGHT

Calling `async def f()` does **nothing** by itself — it just creates a coroutine object sitting there, inert. It only runs when the event loop *awaits/schedules* it. (Like a recipe card: writing it doesn't cook anything; a chef has to pick it up.)

TIES BACK TO L1

“`await` pauses my task and lets others run” works *because* coroutines are pausable functions. Now you know the gears inside the event loop.

CHEAT-SHEET

- Generator = a function that pauses at `yield`, produces values lazily (one at a time).
- It remembers its place & variables between resumes.
- Coroutine (`async def`) = same pause/resume machinery, pauses at `await`.
- `yield` hands back **data**; `await` hands back **control**.
- Calling a coroutine returns an inert object — it runs only when awaited/scheduled.
- Generators save memory (lazy); great for streaming large data.

SELF-CHECK

1. What do a generator and a coroutine have in common under the hood?
2. You write `result = my_async_fn()` and nothing happens. Why — and what's missing?
3. When would a generator save you memory vs a normal function returning a list?

1 The problem: things you must clean up

Files, database sessions, network connections, locks — you **open/acquire** them, use them, and **must release** them. If an error happens mid-use and you forget to release, you **leak** resources (connections pile up, the app dies).

📖 ANALOGY – A LIBRARY BOOK

A context manager (`with`) is like borrowing a library book that **returns itself automatically the moment you leave** — even if you trip and fall on the way out (an exception). You literally cannot forget to return it.

THE `WITH` BLOCK GUARANTEES CLEANUP

`with open(f) as file:` → **ACQUIRE** (`__enter__`)

use the resource...

SUCCESS

ERROR

RELEASE (`__exit__`)

RELEASE anyway (`__exit__`)

2 `async with` for async resources

Async resources (a DB session, an HTTP client) use `async with` (backed by `__aenter__` / `__aexit__`). You'll use this constantly in Stage 0.5 for the SQLAlchemy session — open a session, do work, and it commits/closes safely even on error.

💡 TWO WAYS TO MAKE ONE

A class with `__enter__` / `__exit__`, or the easy way: decorate a generator with `@contextlib.contextmanager` — the code before `yield` is “acquire,” after is “release.” (Notice: generators show up again!)

CHEAT-SHEET

- `with` guarantees cleanup runs — even if an exception is thrown.
- Mechanism: `__enter__` (acquire) → body → `__exit__` (release, always).
- `async with` = the async version (`__aenter__` / `__aexit__`) for DB sessions, connections.
- Build one via a class, or `@contextmanager` on a generator.
- Use for: files, DB sessions, locks, connections — anything you must release.

SELF-CHECK

1. Why is `with open(...)` safer than opening a file and calling `.close()` yourself?
2. What runs in `__exit__`, and when is it guaranteed to run?
3. Why would a DB session use `async with` instead of plain `with`?

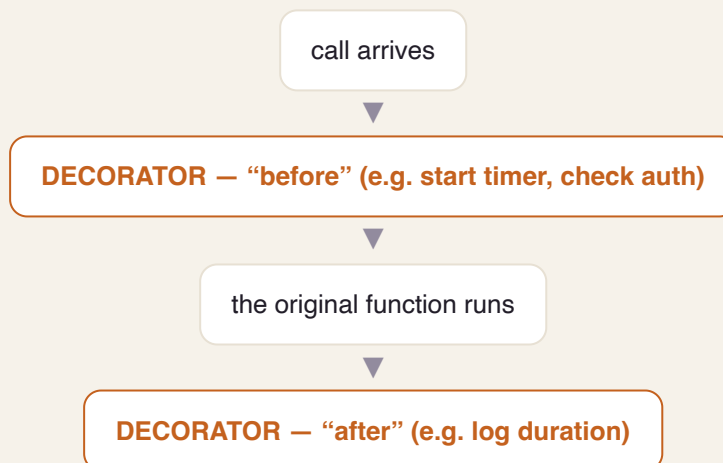
1 The idea: gift-wrapping a function

In Python, functions are just objects you can pass around. A **decorator** takes a function, **wraps it in an extra layer** that runs code before/after, and returns the wrapped version — *without changing the original function's code*.

📁 ANALOGY – GIFT WRAPPING

The gift inside is unchanged; you add a layer around it. Decorators add **cross-cutting behavior** — logging, timing, auth checks, retries, caching — the same way, to many functions, without copy-pasting that logic into each.

A DECORATOR WRAPS THE CALL



2 You'll see these everywhere

`@app.get("/jobs")` (FastAPI routes), `@pytest.fixture` (tests), `@property`, `@lru_cache` — all decorators. Tip: use `functools.wraps` inside your decorator so the wrapped function keeps its real name and docs (otherwise debugging gets confusing).

🔗 YOUR DEVOPS BRIDGE

A decorator is basically **middleware** for a single function — the same “run something before/after” idea as nginx/Envoy filters or HTTP middleware, at function scope.



CHEAT-SHEET

- Functions are objects → you can wrap them.
- A decorator takes a function, returns a wrapped one that adds before/after behavior.
- `@name` above a function applies the decorator.
- Great for cross-cutting concerns: logging, timing, auth, retry, caching.
- Use `functools.wraps` to preserve the original name/docs.
- FastAPI & pytest are built on decorators.



SELF-CHECK

1. In one sentence, what does a decorator do to a function?
2. Name two cross-cutting concerns that are perfect for a decorator, and why.
3. Why bother with `functools.wraps`?

1 Two containers for structured data

Both hold structured data with named, typed fields. The difference is **whether they validate**:

TRUSTED CORE VS UNTRUSTED EDGE (TIES TO L2)

@DATACLASS – PLAIN BOX

Auto-generates `__init__` / `__repr__` / `__eq__`. **No validation**. Use for **trusted, internal** data you already checked. Fast, zero-dependency. `frozen=True` makes it immutable.

PYDANTIC MODEL – BOX + INSPECTOR

Same fields, but **validates & coerces at construction**, and serializes to/from JSON. Use at the **boundary** (API input, config, external data).

ANALOGY

A **dataclass** is a labeled box you pack yourself (you trust the contents). A **Pydantic model** is a box with a customs inspector who checks everything going in. Don't pay for the inspector on internal boxes you already trust.

RULE OF THUMB

Untrusted input crosses a boundary → **Pydantic**. Internal, already-validated domain object → **dataclass**. (This is the “validate at the edges, trust the core” pattern from L2, made concrete.)

CHEAT-SHEET

- Both = typed, named-field containers.
- `@dataclass`: no validation, fast, for trusted internal data; `frozen=True` = immutable.
- Pydantic: validates + coerces + (de)serializes; for untrusted boundary data.
- Pick by *where the data comes from*: edge → Pydantic, core → dataclass.

✅ **SELF-CHECK**

1. Your worker passes an already-validated job object between two internal functions. Dataclass or Pydantic?
2. Why not just use Pydantic for everything?
3. What does `frozen=True` buy you?

1 Why tests (the real reason)

Tests aren't about proving code works today — they're a **safety net** that lets you change code tomorrow *without fear*. For a long-lived platform, that confidence is everything.

ANALOGY

Tests are **smoke alarms** throughout the house. You don't notice them daily, but the moment you break something, one goes off — right next to the problem, not after the house burns down in production.

2 The pytest toolkit

THE FOUR TOOLS YOU'LL USE CONSTANTLY

FIXTURES

Reusable **setup/teardown** run before/after tests (a DB, a client, sample data). The “prep kitchen” that lays out ingredients so each test starts clean.

PARAMETRIZE

Run the **same test with many inputs** automatically — one test, a table of cases (a spice rack of inputs).

MOCKING

Replace slow/external things (DB, Claude API) with a **stunt double** so tests are fast, deterministic, and free.

ASYNC TESTS

`pytest-asyncio` lets you `await` inside tests — needed to test your async code.

3 Hypothesis — the robot that breaks your code

Normal tests check examples *you* think of. **Property-based testing** (Hypothesis) flips it: you state a **property that must always hold** (“encoding then decoding returns the original”), and Hypothesis throws **hundreds of weird inputs** at it to find a counterexample — then shrinks it to the smallest failing case.

💡 WHY IT'S POWERFUL

It finds the edge cases you'd never think to write — empty strings, huge numbers, unicode, zero. Great for parsers, encoders, and pure logic.

📝 CHEAT-SHEET

- Tests = a safety net for fearless change, not busywork.
- **fixtures** = setup/teardown & dependency injection for tests.
- **parametrize** = same test, many inputs.
- **mocking** = fake the slow/external stuff (DB, APIs).
- `pytest-asyncio` = test async code.
- **Hypothesis** = state a property; it hunts counterexamples.
- Test pyramid: many fast unit tests, fewer integration, fewest end-to-end.

✅ SELF-CHECK

1. What's the real value of a test suite for a system you'll change for months?
2. When you test a function that calls the Claude API, what do you do to that call, and why?
3. Give one property (not an example) you could hand to Hypothesis for a “parse job id” function.

1 The three pieces

ENGINE · SESSION · ORM

ENGINE

The **phone line** to the database — a **connection pool** it reuses (opening a new DB connection per request is expensive).

SESSION (UNIT OF WORK)

Your **shopping cart**: add/change/delete objects, then `commit()` applies them all in one **transaction** (or `rollback()` undoes everything).

ORM

A **translator** between Python objects and SQL rows — you work with a `Job` object, it writes the SQL.

ANALOGY

The session is a shopping cart: you toss items in as you shop, and **nothing is actually bought until checkout** (`commit`). If you change your mind, empty the cart (`rollback`) — the store shelves (DB) are untouched.

2 Why *async* here matters

Use the **async driver** (`asyncpg`) with `AsyncSession` and `async with`. Reason: a DB query is I/O — with a *sync* driver it would **block the event loop** (Lesson 1's cardinal sin!) and freeze every other request. The async driver lets the loop serve others while the DB works.

TIES TO STAGE 0

You'll hide the session behind a **repository** (e.g., `JobRepository.get(id)`) so your business logic never touches SQL directly — that's the “ports & adapters” idea you'll formalize in Stage 0.

⚠️ WATCH OUT

The **N+1 query problem**: looping over 100 jobs and lazily loading each one's rows = 101 queries. Load them together (eager loading) instead. Classic performance trap.

📝 CHEAT-SHEET

- Engine = connection pool (reused DB connections).
- Session = unit of work / shopping cart; `commit` = checkout, `rollback` = undo.
- ORM = maps Python objects ↔ SQL rows.
- Use the **async driver** (`asyncpg`) + `async with` so DB calls don't block the loop.
- Hide it behind a **repository** (keeps SQL out of business logic).
- Beware the **N+1** query trap — load related rows together.

✅ SELF-CHECK

1. Why must a DB call in an async app use an async driver? (tie it to Lesson 1)
2. What does `commit()` vs `rollback()` do, in the shopping-cart analogy?
3. What is the N+1 problem, and how do you avoid it?

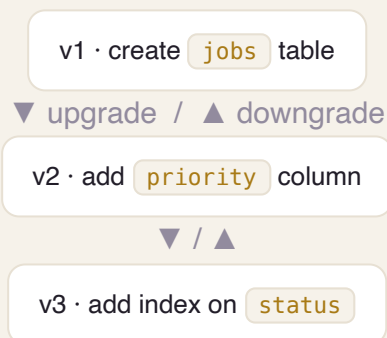
1 The problem: the schema must change over time

You'll add columns, tables, indexes as the app grows. But the database already has **real data** — you can't just drop and recreate it. You need **ordered, repeatable, reversible** changes that every environment (your laptop, staging, prod) applies identically.

ANALOGY — GIT FOR YOUR SCHEMA

Alembic is **version control for the database structure**. Each migration is like a commit: a numbered step with an `upgrade()` (go forward) and `downgrade()` (roll back). The chain of migrations = the schema's history.

A CHAIN OF ORDERED, REVERSIBLE STEPS



2 Autogenerate — with a warning

Alembic can **compare your ORM models to the actual DB** and auto-write a candidate migration. Huge time-saver — but **always read what it generated** before applying (it can miss renames or generate destructive steps). Golden rule: **never edit a migration that has already shipped** — add a new one.

CHEAT-SHEET

- Migration = a versioned, ordered, reversible schema change (like a git commit).
- `upgrade()` = apply; `downgrade()` = revert.
- Autogenerate diffs models vs DB → candidate migration (always review it!).
- Run migrations in deploy/CI so every environment matches.
- Never edit an already-shipped migration — write a new one.

✅ **SELF-CHECK**

1. Why can't you just recreate the database when the schema changes?
2. What are `upgrade()` and `downgrade()` for?
3. Why must you review an auto-generated migration before running it?

Project Hygiene — uv, venv, pyproject

reproducible, isolated
projects

1 Virtual environments — isolation

A **virtual environment** is a private, isolated set of installed packages *per project*, so Project A's libraries never clash with Project B's (or your system Python).

ANALOGY

A venv is a **clean, dedicated workshop** for one project — its own tools on the bench. You don't share one messy garage across every project and hope nothing conflicts.

2 uv, lockfiles & pyproject.toml

uv is a fast, modern tool that creates the venv, installs dependencies, and (crucially) writes a **lockfile** — the exact pinned versions of every package. That makes installs **reproducible**: your machine, a teammate's, and CI all build the *identical* environment.

`pyproject.toml` is the project's spec sheet (name, deps, dev-deps, tooling config).

WHY THE LOCKFILE MATTERS

“Works on my machine” bugs usually mean different package versions. A lockfile makes the environment **exact and repeatable everywhere** — the same reproducibility instinct you already have from Docker images and Terraform.

CHEAT-SHEET

- venv = isolated packages per project (no cross-project clashes).
- `uv` = fast tool for venv + install + lockfile (modern pip/venv replacement).
- Lockfile = exact pinned versions → reproducible installs everywhere.
- `pyproject.toml` = project metadata, dependencies, tool config.
- Separate runtime deps from dev deps (pytest, mypy).

✅ **SELF-CHECK**

1. What problem does a virtual environment solve?
2. What does a lockfile guarantee that a loose dependency list doesn't?
3. Which of your DevOps habits does this map onto?